Logo UTP

Universidad Tecnológica de Puebla

Tecnologías de la Información, área Desarrollo de Software Multiplataforma

Materia: Desarrollo de Aplicaciones Web Progresivas

SUBPRODUCTO 5

Aportación

Proyecto: AlertaVecinal, red de seguridad vecinal (PWA)

Profesora: [NOMBRE_PROFESORA]

Grupo: 10 D

Integrantes: Serrano Arana Jahir Jaziel · Matrícula [MATRICULA_1]
Xicali Diaz Luis Alberto · Matrícula [MATRICULA_2]
Gonzalez Barrios Yeudiel Nain · Matrícula [MATRICULA_3]

Fecha: 29 de octubre de 2026

Índice

Sección común: análisis de la aplicación y conversión a PWA 4

1. Funcionalidad actual de la aplicación web 4
2. Limitaciones de la versión web 6
3. ¿Por qué conviene convertirla en PWA? 6

Aportación 1: Alerta sin señal 8

a) Hoja de presentación del estudiante 8
b) Nombre de la aportación 9
c) Categoría 9
d) Problema o necesidad identificada 9
e) Descripción de la propuesta 9
f) Valor agregado 10
g) Elemento diferenciador 10
h) Imágenes 11

Aportación 2: Mapa de calor de incidentes 13

a) Hoja de presentación del estudiante 13
b) Nombre de la aportación 14
c) Categoría 14
d) Problema o necesidad identificada 14
e) Descripción de la propuesta 14
f) Valor agregado 15
g) Elemento diferenciador 15
h) Imágenes 16

Aportación 3: Vecino Verificado Pro 18

a) Hoja de presentación del estudiante 18
b) Nombre de la aportación 19
c) Categoría 19
d) Problema o necesidad identificada 19
e) Descripción de la propuesta 19
f) Valor agregado 21
g) Elemento diferenciador 21
h) Imágenes 22

Referencias 24

URL de la aplicación **25**

Sección común: análisis de la aplicación y conversión a PWA

1. Funcionalidad actual de la aplicación web

AlertaVecinal es una red de seguridad para una colonia o fraccionamiento. Antes de convertirla en PWA, la versión web ya resolvía el flujo completo de una alerta, usando HTML, CSS y JavaScript puro con Firebase como backend:

Función	Cómo funciona	Tecnología
Registro	El vecino escribe nombre, correo, contraseña, calle y número. Su cuenta queda como **pendiente**.	Firebase Authentication + Firestore (`usuarios`)
Inicio y cierre de sesión	Si la cuenta está pendiente o suspendida se muestra un aviso y no se entra al panel.	`onAuthStateChanged` y `onSnapshot` del perfil
Roles	"admin" y "vecino". El primer admin se asigna a mano en Firestore.	Campo `rol` + reglas de seguridad
Administración	Listar usuarios y aprobarlos, suspenderlos, cambiarles el rol o eliminarlos.	`admin.html`, `updateDoc`, `deleteDoc`
Enviar alerta	Tipo, descripción y ubicación GPS, más la dirección legible.	API de geolocalización + Nominatim
Alertas en tiempo real	Lista que se actualiza sola cuando alguien envía una alerta.	Firestore `onSnapshot`
Mapa	Un marcador de color por alerta, con su descripción.	Leaflet + OpenStreetMap
Seguridad	Cada usuario edita solo su perfil, solo el admin gestiona usuarios, solo los aprobados crean alertas.	`firestore.rules`

Tabla 1. Funcionalidad de la aplicación web antes de la conversión.

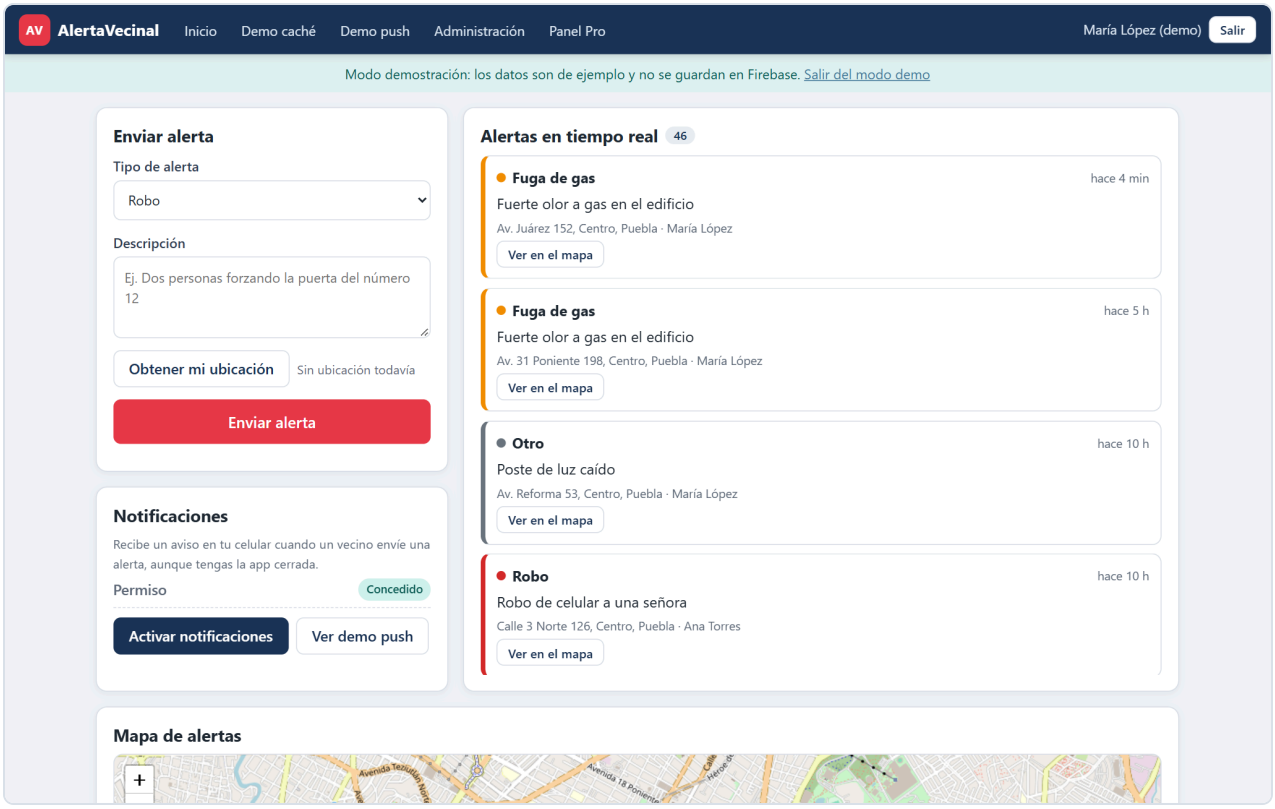


Figura 1. Panel del vecino en escritorio: enviar alerta, notificaciones y lista en tiempo real (datos de ejemplo).

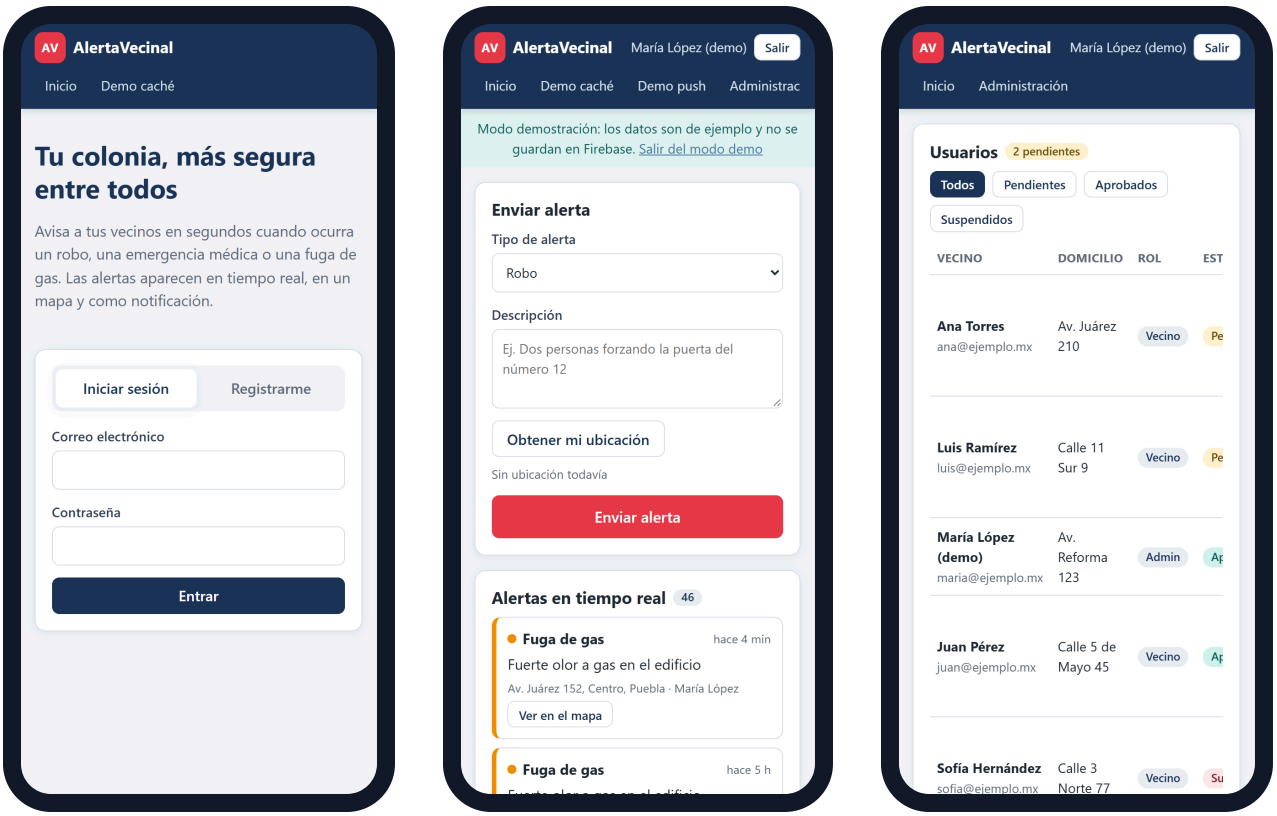


Figura 2. La misma aplicación en celular: acceso, panel y administración. El diseño es responsivo.

2. Limitaciones de la versión web

Aunque la app funcionaba, al probarla pensando en una emergencia real encontramos problemas que una página web normal no puede resolver:

- **Depende por completo de la red.** Con señal débil la página tarda en cargar y sin señal no abre.
- **No se puede instalar.** El vecino debe recordar la dirección y abrir el navegador, lo que cuesta segundos valiosos.
- **No avisa con la app cerrada.** Una alerta solo se ve si el vecino tiene la página abierta en ese momento.
- **Una alerta enviada sin conexión se pierde.** El formulario falla y el vecino puede creer que avisó.
- **El mapa se vuelve a descargar** cada vez, gastando datos móviles.

3. ¿Por qué conviene convertirla en PWA?

Una Aplicación Web Progresiva conserva el mismo código web, pero agrega capacidades de app nativa sin pasar por tiendas de aplicaciones ni pedirle al vecino que descargue un archivo pesado. Cada limitación tiene una respuesta:

Limitación de la web	Solución PWA	Archivo
No abre sin señal	Service worker con caché del app shell y página `offline.html`	`sw.js`, `offline.html`
Hay que buscarla en el navegador	Manifiesto + botón "Instalar app": ícono en la pantalla de inicio y ventana propia	`manifest.json`, `js/instalar.js`
No avisa con la app cerrada	Notificaciones push con Firebase Cloud Messaging	`sw-push.js`, `js/push.js`
Alertas perdidas sin conexión	Cola en IndexedDB y Background Sync (Aportación 1)	`js/offline-sync.js`, `sw-sync.js`
El mapa gasta datos	Mosaicos con Stale-While-Revalidate en una caché limitada	`sw.js`

Tabla 2. Problemas de la versión web y cómo los resuelve la PWA.

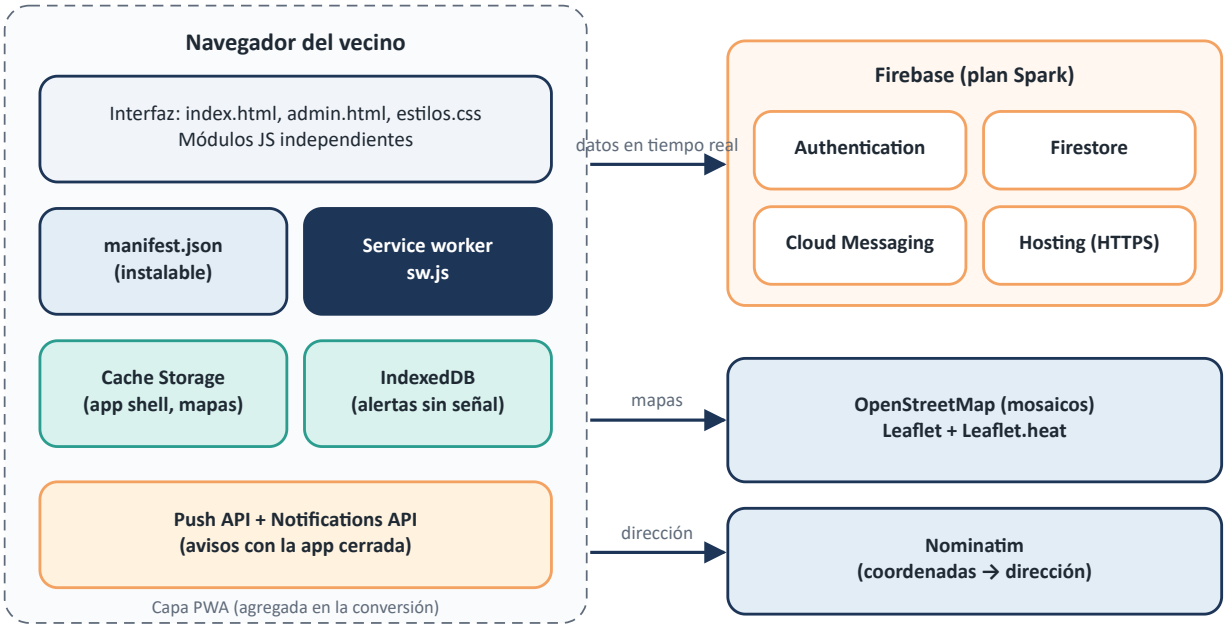


Figura 3. Arquitectura de AlertaVecinal: la capa PWA dentro del navegador y los servicios externos. Diagrama elaborado por el equipo.

Además, todo se mantiene en el plan gratuito **Spark** de Firebase: no se usan Cloud Functions, y el envío de notificaciones se hace con un script de Node en la computadora del administrador. Con esta base, cada integrante propuso una mejora que aprovecha las capacidades de la PWA desde un ángulo distinto: la confiabilidad, la información y la sostenibilidad del proyecto.

Aportación 1: Alerta sin señal

a) Hoja de presentación del estudiante

APORTACIÓN INDIVIDUAL 1

Alerta sin señal

Estudiante	Serrano Arana Jahir Jaziel
Matrícula	[MATRICULA_1]
Grupo	10 D
Carrera	Tecnologías de la Información, área Desarrollo de Software Multiplataforma
Materia	Desarrollo de Aplicaciones Web Progresivas
Profesora	[NOMBRE_PROFESORA]
Categoría	Innovación
Fecha	29 de octubre de 2026

Universidad Tecnológica de Puebla · Proyecto AlertaVecinal

b) Nombre de la aportación

Alerta sin señal: envío diferido y automático de alertas creadas sin conexión.

c) Categoría

Innovación. La mejora no cambia lo que hace la app, sino cómo lo hace: introduce una técnica (almacenamiento local con sincronización en segundo plano) que no existía en la versión web y que cambia la confiabilidad del servicio.

d) Problema o necesidad identificada

Las emergencias no esperan a que haya buena señal. Un robo puede ocurrir en un estacionamiento subterráneo, una fuga de gas en un cuarto de servicio y una emergencia médica en una casa donde el módem se quedó sin luz. También hay vecinos que se quedan sin saldo de datos a mitad de mes.

En la versión web, si el vecino pulsaba "Enviar alerta" sin conexión, la petición a Firestore fallaba. En el mejor caso aparecía un mensaje de error; en el peor, el vecino cerraba la app pensando que ya había avisado. El riesgo no es solo técnico: una alerta perdida da una **falsa sensación de seguridad** justo cuando más importa.

e) Descripción de la propuesta

Cuando no hay conexión, la alerta ya no se intenta enviar: se guarda en el teléfono y se manda sola cuando regresa la señal. El vecino ve un aviso amarillo que dice cuántas alertas están esperando. Funciona en dos caminos para cubrir todos los navegadores:

1. **Background Sync (Chrome, Edge y Android).** Al guardar la alerta se registra una sincronización con la etiqueta `enviar-alertas`. Cuando el sistema detecta conexión, despierta al service worker aunque la app esté cerrada, y `sw-sync.js` envía la alerta a Firestore con su API REST y el token de sesión del vecino.
2. **Respaldo con el evento `online` (Firefox y Safari).** Estos navegadores no tienen Background Sync, así que la página escucha el regreso de la conexión y envía las alertas pendientes con el SDK de Firebase.

Cada alerta guardada recibe un **identificador propio** desde el principio. Si los dos caminos intentan enviarla, la segunda escritura encuentra el documento ya creado y la alerta no se duplica. Las alertas que llegan por esta vía se marcan con `origen: "sin-senal"` para que los vecinos sepan que fueron enviadas con retraso.

Archivo **`js/offline-sync.js`**, líneas 42 a 57

```
export async function encolar(datos) {
  // Id propio: si el SW y la página intentan enviarla, no se duplica.
  const id = 'sin-senal-' + crypto.randomUUID().replaceAll('-', '').slice(0, 16);
  // Token de sesión vigente (Firebase lo tiene en memoria aunque no haya red).
  // El SW lo usa para escribir en Firestore por su API REST.
  const idToken = !esDemo && auth.currentUser ? await auth.currentUser.getIdToken().catch(() =>
null) : null;
  await guardar({ id, datos, idToken, guardada: Date.now() });

  // Pedimos al navegador un Background Sync, si lo soporta.
  const registro = await (window.AV.swListo || Promise.resolve(null));
  if (!esDemo && registro && 'sync' in registro) {
    await registro.sync.register(ETIQUETA_SYNC);
  }
  aviso('Sin señal: tu alerta se guardó en el teléfono y se enviará sola al volver la
conexión.', 'info', 6000);
  await pintarPendientes();
}
```

Archivo **sw-sync.js**, líneas 80 a 84

```
self.addEventListener('sync', (event) => {
  if (event.tag === 'enviar-alertas') {
    event.waitUntil(enviarPendientesDesdeSW());
  }
});
```

La mejora se integra sin tocar el código de los demás integrantes: `alertas.js` solo pregunta si existe `window.AV.offline`; si el módulo no está cargado, la app funciona como antes.

f) Valor agregado

- **Ninguna alerta se pierde:** lo que el vecino escribe queda guardado aunque cierre la app o se apague la pantalla.
- **Menos pasos en un momento de estrés:** no hay que recordar reintentar; el envío es automático.
- **Transparencia:** el aviso "esperando señal" le dice al vecino la verdad sobre el estado de su alerta.
- **Sin costo extra:** usa capacidades del navegador y del plan gratuito de Firebase.

g) Elemento diferenciador

Un formulario web común falla sin conexión y obliga a volver a escribir todo. La propuesta combina tres elementos que no suelen verse juntos en un proyecto escolar: **persistencia local en IndexedDB**, **envío desde el service worker con la app cerrada** y un **respaldo para los navegadores que no soportan Background Sync**, todo con protección contra duplicados. Además, la alerta conserva la hora real en que el vecino la creó, no la hora en que volvió la señal.

h) Imágenes

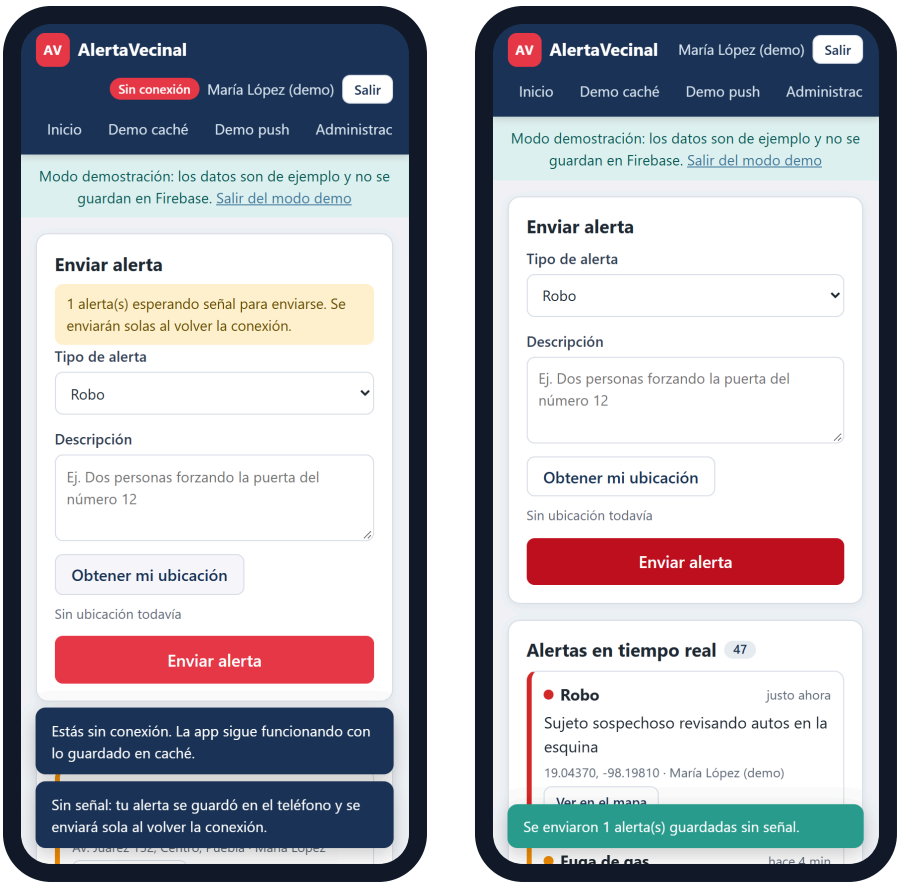


Figura 4. Izquierda: sin conexión, la alerta queda "esperando señal". Derecha: al volver la conexión se envió sola y aparece primera en la lista. Capturas reales tomadas con Playwright.

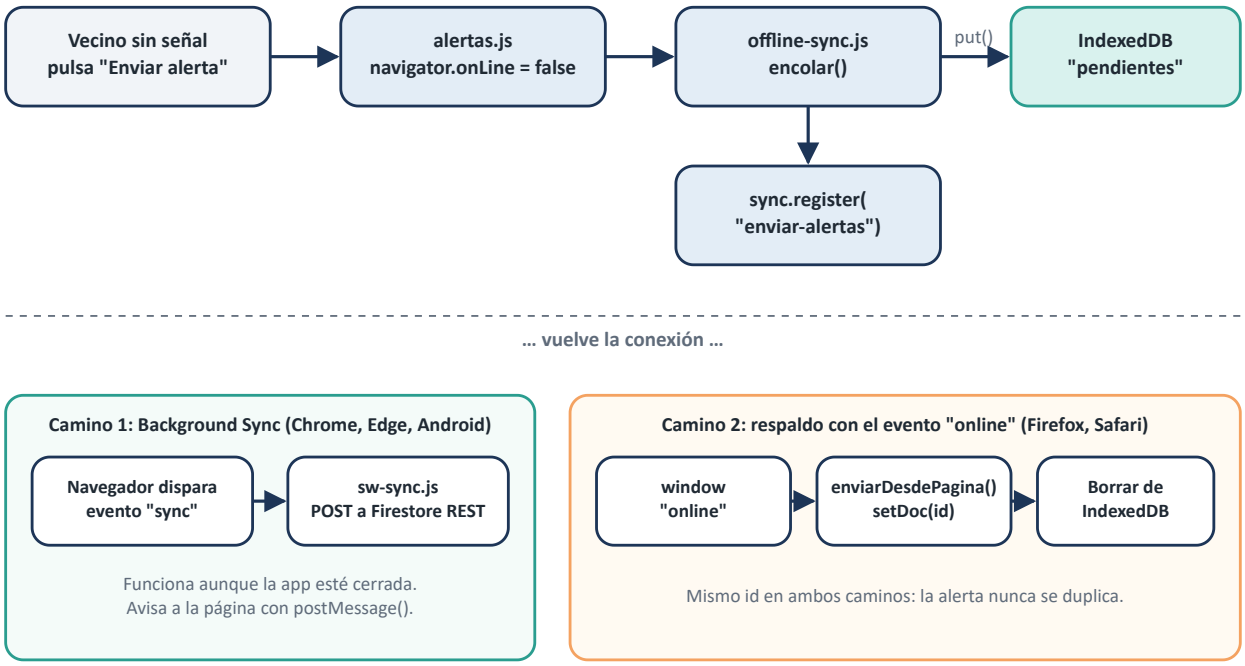


Figura 5. Flujo de la aportación Alerta sin señal, con sus dos caminos de envío. Diagrama elaborado por el equipo.

Aportación 2: Mapa de calor de incidentes

a) Hoja de presentación del estudiante

APORTACIÓN INDIVIDUAL 2

Mapa de calor de incidentes

Estudiante	Xicali Diaz Luis Alberto
Matrícula	[MATRICULA_2]
Grupo	10 D
Carrera	Tecnologías de la Información, área Desarrollo de Software Multiplataforma
Materia	Desarrollo de Aplicaciones Web Progresivas
Profesora	[NOMBRE_PROFESORA]
Categoría	Creatividad + innovación
Fecha	29 de octubre de 2026

Universidad Tecnológica de Puebla · Proyecto AlertaVecinal

b) Nombre de la aportación

Mapa de calor de incidentes: visualización de las zonas con más alertas, filtrable por tipo y por fecha.

c) Categoría

Creatividad e innovación. Es creativa porque presenta los mismos datos de otra forma, más visual y fácil de entender para cualquier vecino. Es innovadora porque agrega una capa de análisis que la app no tenía, calculada en el propio navegador y en tiempo real.

d) Problema o necesidad identificada

La lista y el mapa de marcadores responden a la pregunta "¿qué acaba de pasar?", pero no a "¿dónde pasa más seguido?". Con unas cuantas decenas de alertas, los marcadores se enciman y ya no se distinguen. Un vecino que quiere decidir por qué calle caminar de noche, o un comité que quiere pedir más alumbrado o rondines de patrulla, necesita ver **patrones**, no puntos sueltos. Esa información ya existía en Firestore, pero nadie podía leerla de un vistazo.

e) Descripción de la propuesta

Se agregó al panel una tarjeta con un segundo mapa que usa el plugin **Leaflet.heat**. Cada alerta se convierte en un punto de calor; donde se juntan muchas, el color pasa de verde a amarillo, naranja y rojo. Los robos y las fugas de gas pesan un poco más que el resto porque representan un riesgo mayor para la zona.

El vecino puede filtrar por **tipo de alerta** y por **rango de fechas** (por defecto, los últimos 30 días). Debajo del mapa aparece un resumen en texto con el número de alertas del periodo y las **tres calles más reportadas**, que se obtienen de la dirección que devuelve Nominatim. Como el módulo escucha el evento `av:alertas`, el mapa se recalcula solo cada vez que alguien envía una alerta nueva.

Archivo **`js/heatmap.js`**, líneas 66 a 76

```
function filtrar() {
  const tipo = document.getElementById('calor-tipo').value;
  const desde = new Date(document.getElementById('calor-desde').value + 'T00:00:00');
  const hasta = new Date(document.getElementById('calor-hasta').value + 'T23:59:59');
  return alertas.filter((a) => {
    if (typeof a.lat !== 'number') return false;
    if (tipo !== 'todos' && a.tipo !== tipo) return false;
    const f = aFecha(a.creado);
    return f >= desde && f <= hasta;
  });
}
```

Archivo **`js/heatmap.js`**, líneas 83 a 97

```
function actualizar() {
  if (!mapaCalor) return;
  const visibles = filtrar();
  // Cada punto: [lat, lng, intensidad]. Robo y gas pesan un poco más.
  const puntos = visibles.map((a) => [a.lat, a.lng, a.tipo === 'robo' || a.tipo === 'gas' ? 1 :
0.7]);
  capaCalor.setLatLngs(puntos);

  const conteo = {};
  visibles.forEach((a) => { const c = calleDe(a.direccion); conteo[c] = (conteo[c] || 0) + 1;
});
  const top = Object.entries(conteo).sort((a, b) => b[1] - a[1]).slice(0, 3);
  document.getElementById('calor-resumen').textContent = visibles.length
    ? `${visibles.length} alertas en el periodo. Calles más reportadas: ` +
      top.map(([c, n]) => `${c} (${n})`).join(', ') + '.'
    : 'No hay alertas con esos filtros.';
}
```

f) Valor agregado

- **Convierte registros en conocimiento:** en segundos se identifican las calles y los horarios que requieren más cuidado.
- **Prevención:** el vecino puede elegir rutas más seguras y el comité puede priorizar dónde poner cámaras o luminarias.
- **Evidencia para gestionar:** el mapa filtrado por robos de un mes es un argumento claro ante la autoridad municipal.
- **Ligero:** no requiere servidor ni base de datos adicional; todo se calcula en el navegador con los datos que ya se reciben.

g) Elemento diferenciador

Los mapas delictivos que publican algunas instituciones suelen estar agregados por colonia o municipio y se actualizan cada mes o cada año. Este mapa está construido con los reportes de los **propios vecinos**, tiene detalle a nivel de **calle**, se actualiza en **tiempo real** y permite combinar filtros de tipo y fecha. Además reutiliza la caché de mosaicos de la PWA, así que también se puede consultar con mala señal.

h) Imágenes

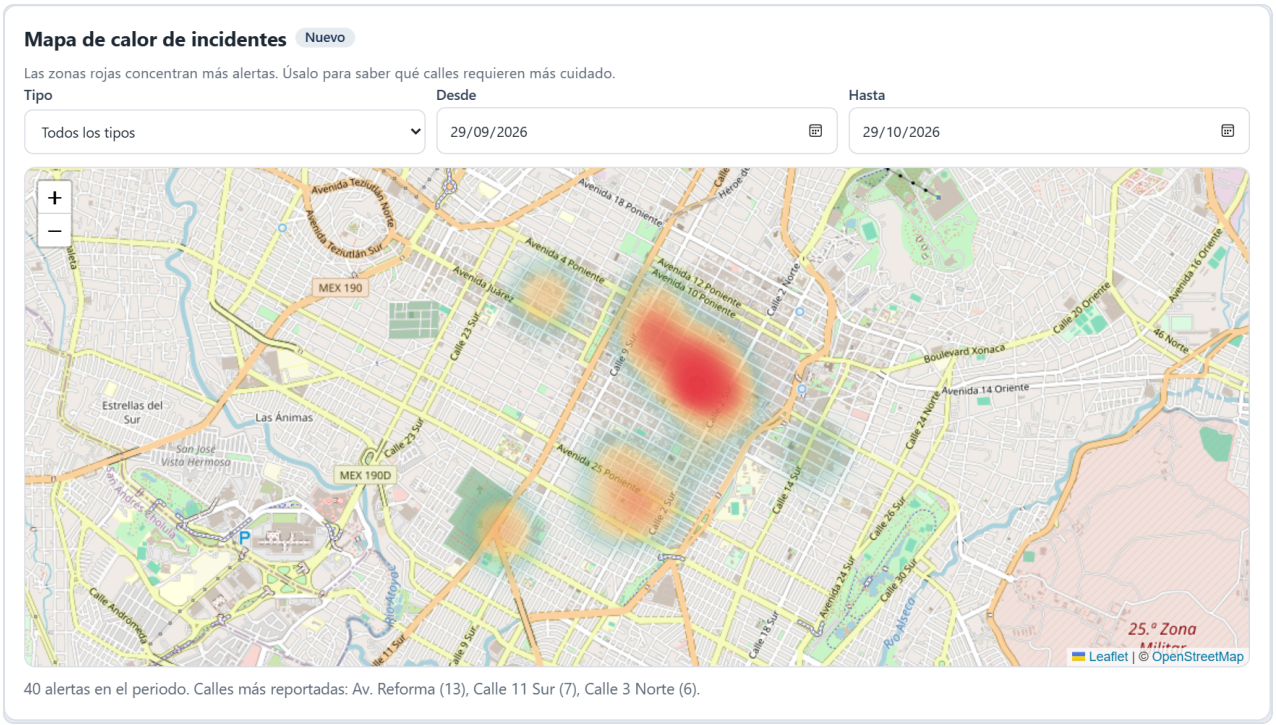


Figura 6. Mapa de calor con todos los tipos de alerta de los últimos 30 días y resumen de calles (datos de ejemplo).

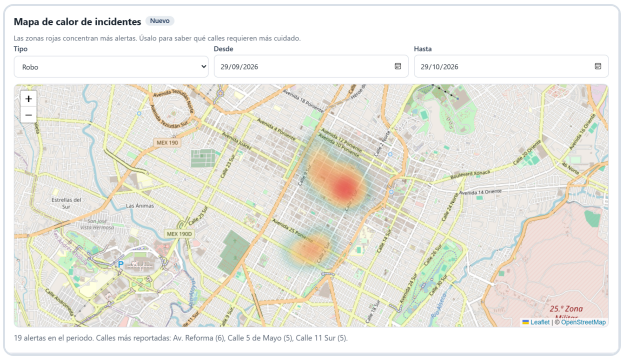


Figura 7. El mismo mapa filtrado solo por robos.



Vista en celular del mapa de calor.

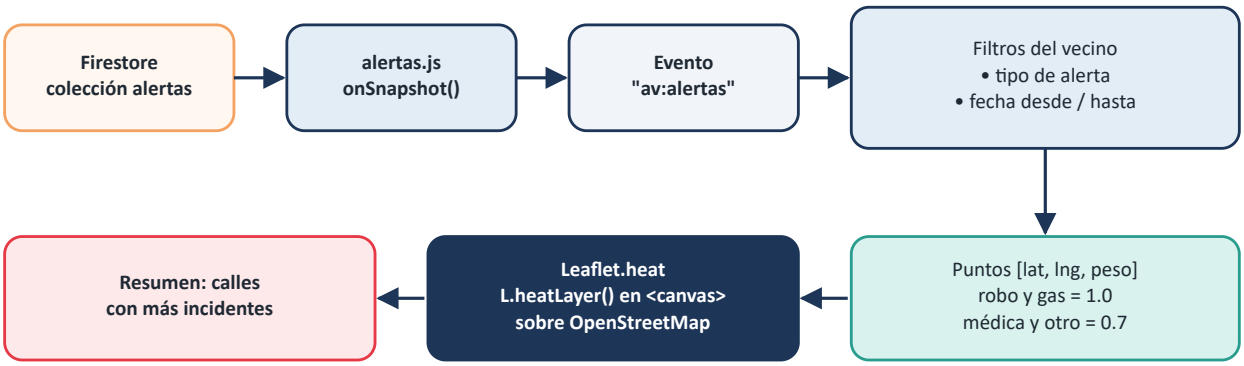


Figura 8. Flujo de datos del mapa de calor: de Firestore al lienzo de Leaflet.heat. Diagrama elaborado por el equipo.

Aportación 3: Vecino Verificado Pro

a) Hoja de presentación del estudiante

APORTACIÓN INDIVIDUAL 3

Vecino Verificado Pro

Estudiante	Gonzalez Barrios Yeudiel Nain
Matrícula	[MATRICULA_3]
Grupo	10 D
Carrera	Tecnologías de la Información, área Desarrollo de Software Multiplataforma
Materia	Desarrollo de Aplicaciones Web Progresivas
Profesora	[NOMBRE_PROFESORA]
Categoría	Emprendimiento + innovación
Fecha	29 de octubre de 2026

Universidad Tecnológica de Puebla · Proyecto AlertaVecinal

b) Nombre de la aportación

Vecino Verificado Pro: modelo de negocio con plan gratuito para vecinos y planes de pago para comités vecinales y fraccionamientos.

c) Categoría

Emprendimiento e innovación. Es emprendimiento porque plantea cómo AlertaVecinal puede generar ingresos y sostenerse como servicio. Es innovación porque agrega productos nuevos sobre la misma plataforma: un panel de estadísticas, la conexión con la caseta de vigilancia y reportes mensuales automáticos.

d) Problema o necesidad identificada

Hay dos necesidades distintas. La primera es de los **clientes**: los comités vecinales y las administraciones de fraccionamientos coordinan la seguridad con grupos de chat, llamadas a la caseta y hojas de cálculo. No tienen un registro ordenado de incidentes, no pueden medir si las medidas funcionan y deben preparar a mano los informes para las asambleas.

La segunda es del **proyecto**: mientras sea gratis y pequeño cabe en el plan Spark, pero si crece necesitará el plan de pago de Firebase, soporte y mantenimiento. Sin ingresos, una app de seguridad corre el riesgo de abandonarse justo cuando la comunidad depende de ella.

e) Descripción de la propuesta

Un modelo **freemium**: los vecinos usan gratis todo lo esencial (enviar y recibir alertas, mapa, notificaciones), y pagan las organizaciones que necesitan administrar y analizar la información. Los planes de pago agregan:

- **Panel de estadísticas:** alertas del periodo, tipo más frecuente, horario de mayor riesgo y calles con más incidentes.
- **Alertas a la caseta:** una tableta en la caseta de vigilancia recibe cada alerta de robo o gas al instante.
- **Reporte mensual en PDF** para presentar en la asamblea o ante la autoridad.
- **Vecino verificado:** el administrador confirma el domicilio antes de aprobar la cuenta, lo que reduce las alertas falsas.

Como prueba de concepto se implementó la **pantalla del panel de estadísticas** (`estadisticas.html`), que calcula los indicadores con las alertas reales de Firestore (o con datos de ejemplo en modo demostración). Es una maqueta: no realiza cobros ni envía avisos reales a ninguna caseta.

Archivo **`js/estadisticas.js`**, líneas 39 a 52

```
function pintarHoras(lista) {
  const franjas = new Array(8).fill(0);
  lista.forEach((a) => { franjas[Math.floor(aFecha(a.creado).getHours() / 3)]++; });
  const max = Math.max(1, ...franjas);
  const ancho = 320, alto = 150, col = ancho / 8;
  const barras = franjas.map((n, i) => {
    const h = (n / max) * (alto - 42);
    const color = n === max ? '#e63946' : '#457b9d';
    return `<rect x="${i * col + 6}" y="${alto - 20 - h}" width="${col - 12}" height="${h}"
rx="4" fill="${color}"/>
      <text x="${i * col + col / 2}" y="${alto - 24 - h}" font-size="10" text-anchor="middle"
fill="#1f2933">${n}</text>
      <text x="${i * col + col / 2}" y="${alto - 6}" font-size="9" text-anchor="middle"
fill="#5f6b7a">${String(i * 3).padStart(2, '0')}h</text>`;
  }).join('');
  $('graf-horas').innerHTML = `<svg viewBox="0 0 ${ancho} ${alto}" width="100%" role="img" aria-
label="Alertas por horario">${barras}</svg>`;
}
```

Modelo Canvas

Socios clave

- Firebase (Google)
- OpenStreetMap
- Empresas de vigilancia privada
- Comités vecinales y juntas auxiliares

Actividades clave

- Desarrollo y soporte de la PWA
- Alta y capacitación de comités

Recursos clave

- Plataforma AlertaVecinal
- Datos de incidentes
- Equipo de desarrollo

Propuesta de valor

- Alertas vecinales confiables y gratuitas para cada vecino
- Para el comité: estadísticas, caseta conectada y reportes listos sin trabajo manual
- Comunidad verificada por domicilio

Relación con clientes

- Autoservicio para vecinos
- Acompañamiento en la instalación para comités

Canales

- La propia PWA (sin tiendas)
- Asambleas vecinales
- Redes sociales locales

Segmentos de clientes

- Vecinos (usuarios gratuitos)
- Comités vecinales de colonias
- Fraccionamientos cerrados con caseta

Estructura de costos

- Firebase plan Blaze al crecer
- Tabletas para casetas (en comodato)
- Soporte y mantenimiento

Fuentes de ingresos

- Suscripción mensual o anual por comunidad (no por vecino)
- Instalación de tableta en caseta
- Reportes extraordinarios

Modelo Canvas de Vecino Verificado Pro (basado en Osterwalder y Pigneur, 2010).

Tabla de precios propuesta

Vecino

$0

para siempre

- Enviar y recibir alertas
- Mapa y mapa de calor
- Notificaciones push
- Alerta sin señal

Comité

$299 MXN

al mes, hasta 150 viviendas

- Todo lo del plan Vecino
- Panel de estadísticas
- Reporte mensual en PDF
- Hasta 3 administradores

Fraccionamiento

$799 MXN

al mes, hasta 600 viviendas

- Todo lo del plan Comité
- Tableta en la caseta de vigilancia
- Administradores ilimitados
- Soporte prioritario

> Los precios son una propuesta inicial del equipo para fines académicos. El pago anual equivaldría a 10 meses (2 meses gratis). Antes de lanzar habría que validarlos con entrevistas a comités reales. El costo se reparte entre las viviendas: en un fraccionamiento de 600 casas, el plan cuesta alrededor de $1.33 MXN por vivienda al mes.

f) Valor agregado

- **Sostenibilidad:** los ingresos pagan la infraestructura y el soporte, y permiten que el plan de los vecinos siga siendo gratis.
- **Respuesta más rápida:** el guardia de la caseta se entera al mismo tiempo que los vecinos, sin depender de una llamada.
- **Decisiones con datos:** el comité sabe qué tipo de incidente crece, a qué hora y en qué calle.
- **Ahorro de tiempo:** el reporte mensual se genera solo en lugar de prepararse a mano.

g) Elemento diferenciador

El cobro es **por comunidad, no por vecino**: nadie queda fuera de la red por no pagar, y la decisión de compra la toma quien ya administra la seguridad. Al ser una PWA, la distribución no pasa por tiendas de aplicaciones: se instala desde un enlace o un código QR en la caseta, sin comisiones ni procesos de aprobación, y funciona igual en Android, iPhone y computadora. Finalmente, la verificación de domicilio por el administrador crea una red de confianza que un grupo de chat abierto no ofrece.

h) Imágenes

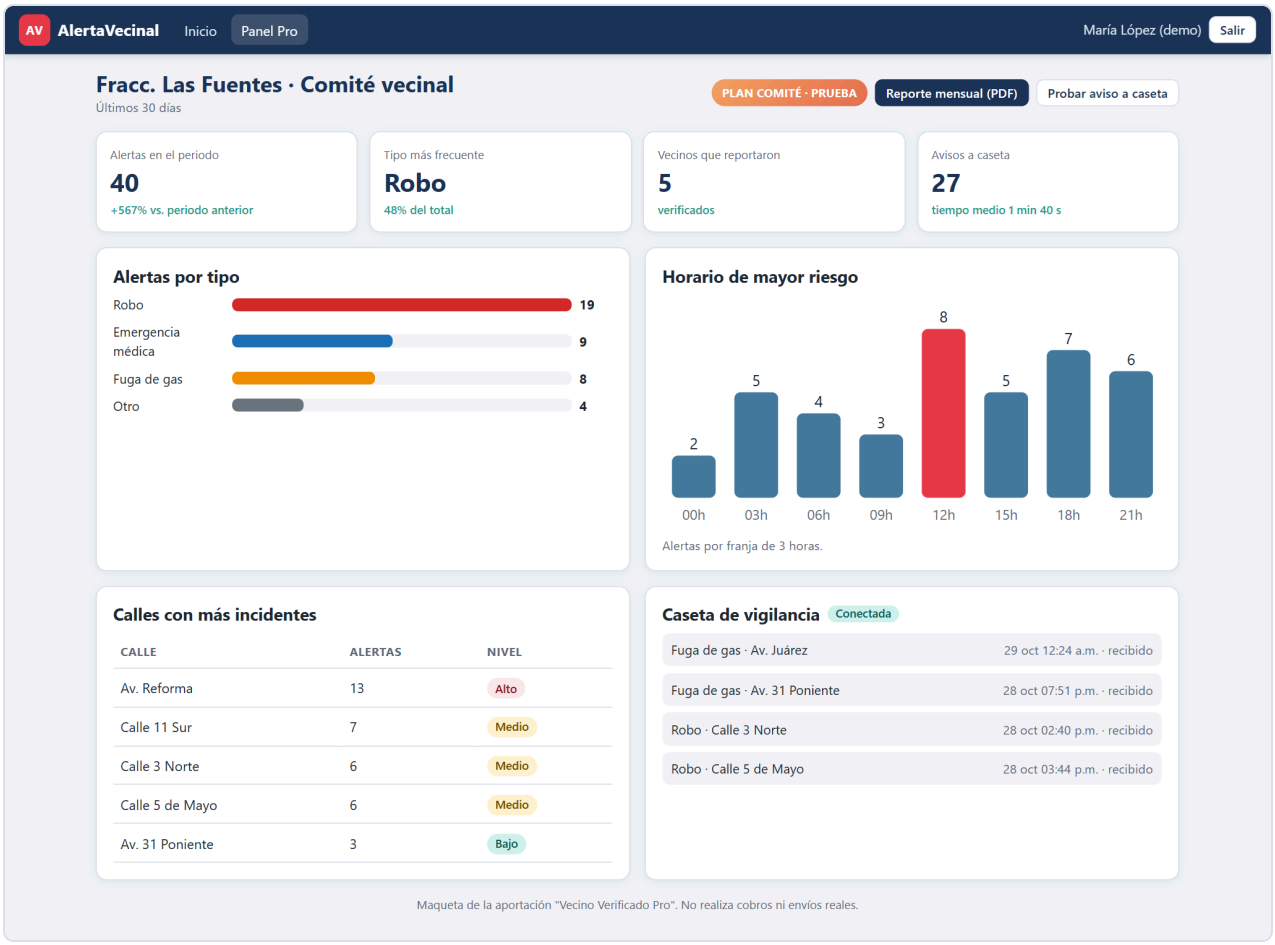


Figura 9. Maqueta funcional del panel de estadísticas del plan Comité (estadisticas.html, datos de ejemplo).



Figura 10. El panel de estadísticas en celular.

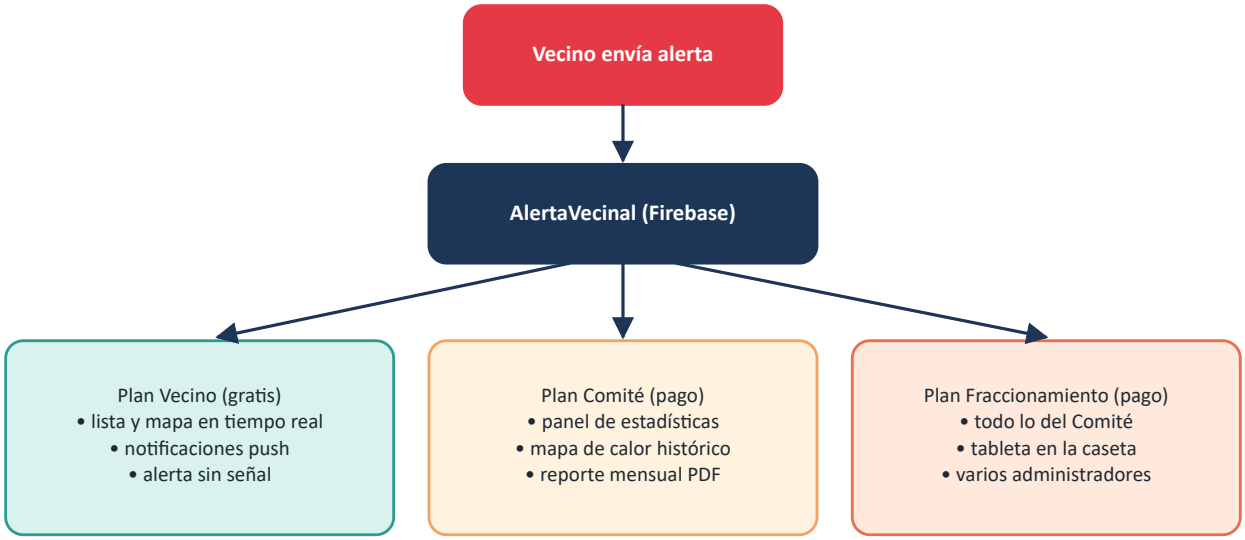


Figura 11. Relación entre la alerta, la plataforma y los tres planes. Diagrama elaborado por el equipo.

Referencias

Archibald, J. (2015). *Introducing background sync*. Chrome for Developers. https://developer.chrome.com/blog/background-sync

Firebase. (s. f.). *Firebase pricing*. Google. https://firebase.google.com/pricing

Leaflet. (s. f.). *Leaflet API reference*. https://leafletjs.com/reference.html

Leaflet. (s. f.). *Leaflet.heat* [Repositorio de código]. GitHub. https://github.com/Leaflet/Leaflet.heat

MDN Web Docs. (s. f.). *Background Synchronization API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API

MDN Web Docs. (s. f.). *IndexedDB API*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API

MDN Web Docs. (s. f.). *Navigator: onLine property*. Mozilla. https://developer.mozilla.org/en-US/docs/Web/API/Navigator/onLine

OpenStreetMap Foundation. (s. f.). *Nominatim usage policy*. https://operations.osmfoundation.org/policies/nominatim/

Osterwalder, A., y Pigneur, Y. (2010). *Business model generation*. John Wiley & Sons.

web.dev. (s. f.). *What makes a good Progressive Web App?* Google. https://web.dev/articles/pwa-checklist

URL de la aplicación

La aplicación AlertaVecinal está publicada en Firebase Hosting y se puede abrir desde cualquier navegador moderno, en computadora o en celular, en la siguiente dirección:

[URL_DE_LA_APP]

Para ver la app con datos de ejemplo, sin iniciar sesión, se puede agregar `?demo=1` al final de la dirección.